Unit3

Unit 3. Transformer architectures for audio

Refresher on transformer models

Using transformers for audio

- 对于 audio model 来说，由于模型本质上都是Transformer，因此它们的架构大部分是共通的，主要区别在于训练和使用方式
- **音频领域四种常见任务**
  - **自动语音识别（ASR）**：输入是语音，输出是文本。
  - **语音合成（TTS）**：输入是文本，输出是语音。
  - **音频分类**：输入是音频，输出是类别概率——序列中每个元素对应一个概率，或者整个序列对应一个总概率。
  - **语音转换或语音增强**：输入和输出均为音频。

处理音频以便将其与 Transformer 结合使用有几种不同的方法。主要考虑因素是：是使用原始形式的音频（即波形），还是将其处理为频谱图。

Model Inputs

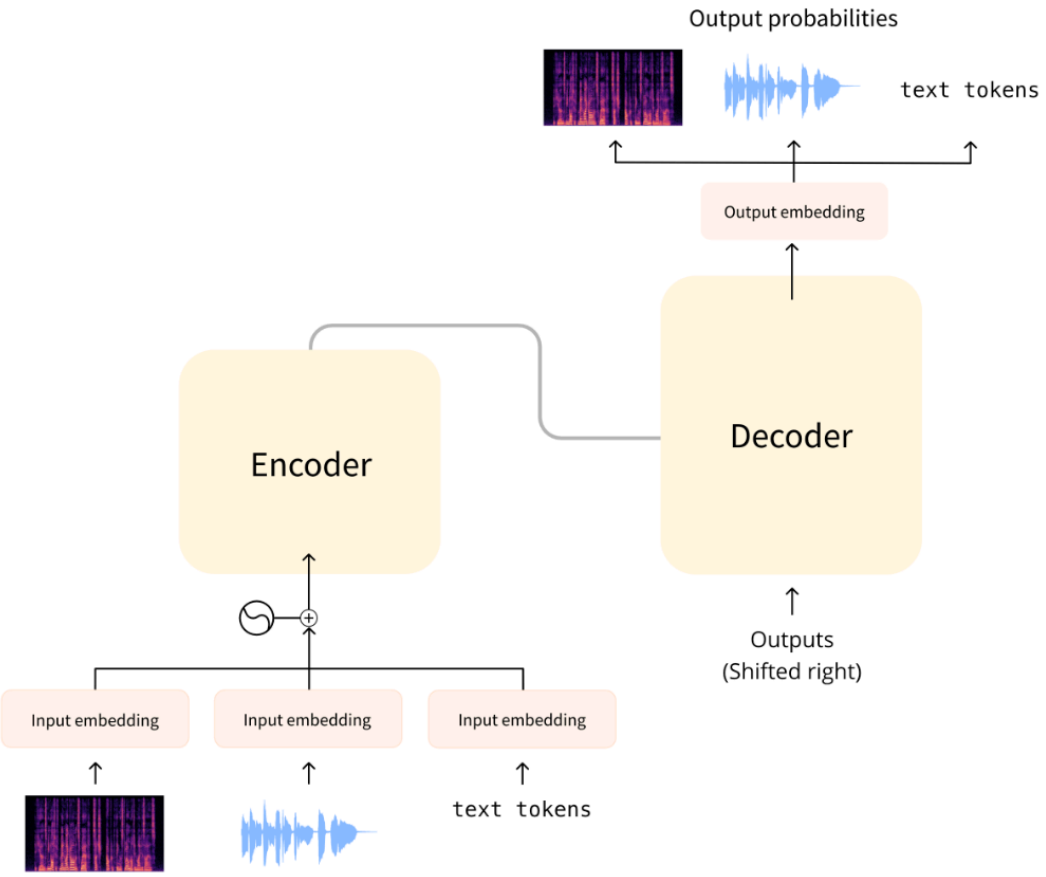

image-165.png

Text Input

- **应用场景**：TTS (Text-to-Speech，文本转语音) 模型。
- **处理逻辑**：输入文本 → 分词 (Tokenize) → Input Embedding Layer → 转换为 512 维向量 → Transformer Encoder。
  - **Embedding 机制**：这与传统 NLP 模型完全一致，目的是将离散的 Token 映射为包含语义特征的连续稠密向量。

Waveform Input

- **应用场景**：ASR (自动语音识别) 模型，例如 Wav2Vec2 和 HuBERT。
- **处理逻辑**：一维连续波形 → 标准化 → 小型 CNN (特征编码器) → 转换为 Embedding → 送入 Transformer。
  - **标准化操作**：在送入网络前，必须将原始波形归一化为**零均值 (Zero mean)** 和**单位方差 (Unit variance)**，这是为了统一不同设备的录音音量和振幅差异，防止模型训练震荡。
  - **特征降维**：CNN 网络通过多层卷积进行下采样 (Subsampling)，不断缩减序列长度。最终的卷积层会将每 25 ms 的音频片段提取并映射为一个 512 维的特征向量。

Spectrogram Input

- **应用场景**：高效 ASR 模型，例如 Whisper。
- **处理逻辑**：原始波形 → 对数梅尔频谱图 (log-mel spectrogram) → 小型 CNN → 转换为 Embedding → 送入 Transformer。
  - **规避长序列灾难**：直接使用原始波形会导致极长的输入序列（例如 16 kHz 采样率下，30 秒音频长度为 $30 \times 16000 = 480000$）。**OOM**
  - **张量重塑**：Whisper 模型固定将音频切分为 30 秒的片段，并生成形状为 `(80, 3000)` 的矩阵（80 代表梅尔频带数，3000 代表时间步长序列）。随后通过 CNN 提取特征进入 Transformer。

Model Outputs

Text Output (文本输出)

- **应用场景**：ASR (自动语音识别)，即把听到的声音转成文字。
- **处理逻辑**：Transformer 输出向量 → 语言模型头 (Language Modeling Head，通常是单线性层) → Softmax 激活函数 → 预测词汇表中的 Token 概率

Spectrogram Output (频谱图输出)

- **应用场景**：TTS (文本转语音)，例如 SpeechT5 模型。

- **处理逻辑**：Transformer 输出向量 → 线性层投影为对数梅尔频谱图 → Post-net (后处理网络，用于降噪) → **Vocoder** → 最终音频波形。

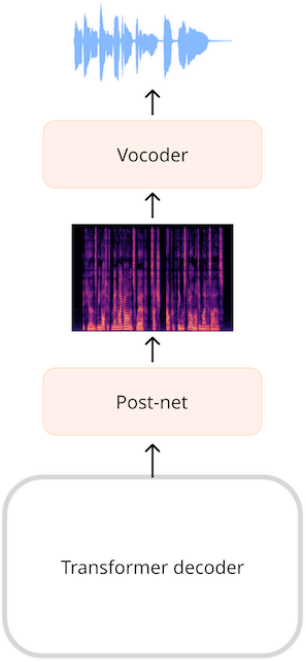

image-166.png

Vocoder 的绝对必要性： 通过 STFT 可以把波形变频谱。理论上做逆运算 (ISTFT) 就能变回波形，**但前提是必须同时有振幅 (Amplitude) 和相位 (Phase) 信息**。

相位缺失问题：音频模型生成的频谱图通常只预测了振幅，没有相位。**Vocoder** 的核心使命，就是去估算并填补这些缺失的相位信息，从而预测完整的 waveform

Waveform Output

- **说明**：理论上模型可以直接一步到位输出波形，绕过频谱图这一中间步骤。 Hugging Face Transformers 库还没有

这部分有新工作吗

CTC architectures

CTC就是将输出与时间同步，一段时间只预测单个字符，同时允许模型在时间轴上产生大量冗余的重复字符和空白符，最后通过 CTC algorithm 合成完整 word

CTC Architecture Basics

- **基本架构**：使用 **Encoder-only transformer** （例如 Wav2Vec2, HuBERT) + 一个额外的**线性分类头 (CTC Head)**

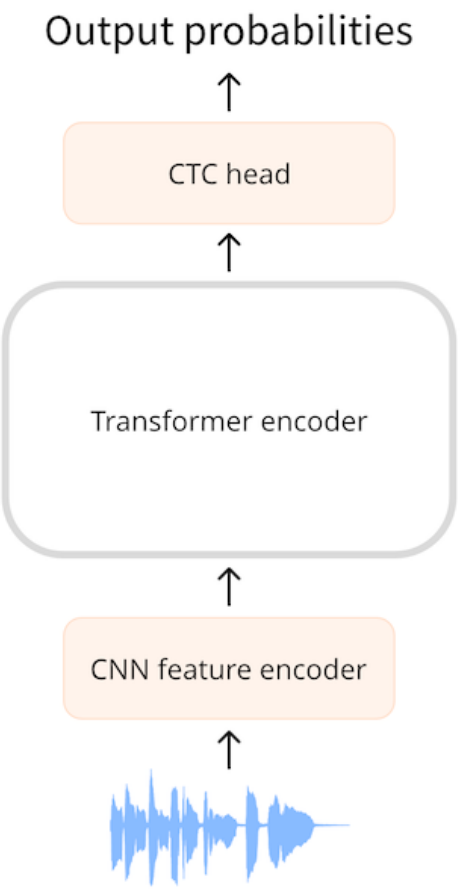

image-167.png

- **预测目标**：与 NLP 动辄几万个词元的庞大词表不同，CTC 模型的词表极小，通常只预测**单个字母/字符**。
  - **预处理细节**：CTC 中，不在乎大小写；数字必须全部拼写为单词（如 `20` 变成 `twenty`）；但必须保留极其重要的标点符号（如英语中的 `'`，因为 `it's` 和 `its` 意思完全不同）

The Alignment Problem

- **核心矛盾**：在 ASR 的训练数据集中，只有 `(整段音频，整句文本)` 的配对，但数据集**没有时间戳信息**将二者一一对应。
- **强制输出导致的重复灾难**：
  - Transformer Encoder 会接收到 50 个特征向量（相当于每 20 毫秒生成一个），模型**必须**对这 50 个向量中的每一个都做出预测（输出一个字符）。
  - **时间维度的不匹配**：由于人类发一个音的时间通常远长于 20 毫秒，模型在连续的多个 20 毫秒切片里听到的都是同一个音。因此，它会连续输出相同的字母，导致结果变成类似 `ERRRRORR` 这样充满重复字符的序列。

The CTC Algorithm

- **核心组件：Blank Token (空白词元 `_`)**：模型在此刻没有听到明确的发音或者充当相同字母之间的硬边界

- **CTC Decoding Logic**：
  1. **合并相邻的重复字符**：除非它们之间被 Blank Token `_` 隔开。
  2. **删除所有的 Blank Token** `_`：得到最终的清晰文本。

Seq2Seq architectures

Seq2Seq Architecture Basics

- **核心定义**：同时使用 Transformer 的 **Encoder (编码器)** 和 **Decoder (解码器)**，将一种序列映射为另一种序列的过程（Sequence-to-Sequence）。
- **任务**：适用于文本摘要或不同语言之间的翻译等 NLP 任务，也适用于语音识别等音频任务。

ℹ **与 CTC 的本质区别**：

- CTC（仅 Encoder）：输入和输出一对一（比如每 20ms 强制吐出一个预测），**输入输出长度强绑定**。
- Seq2Seq：输入序列（音频）和输出序列（文本）的**长度可以完全不同**。

Automatic speech recognition

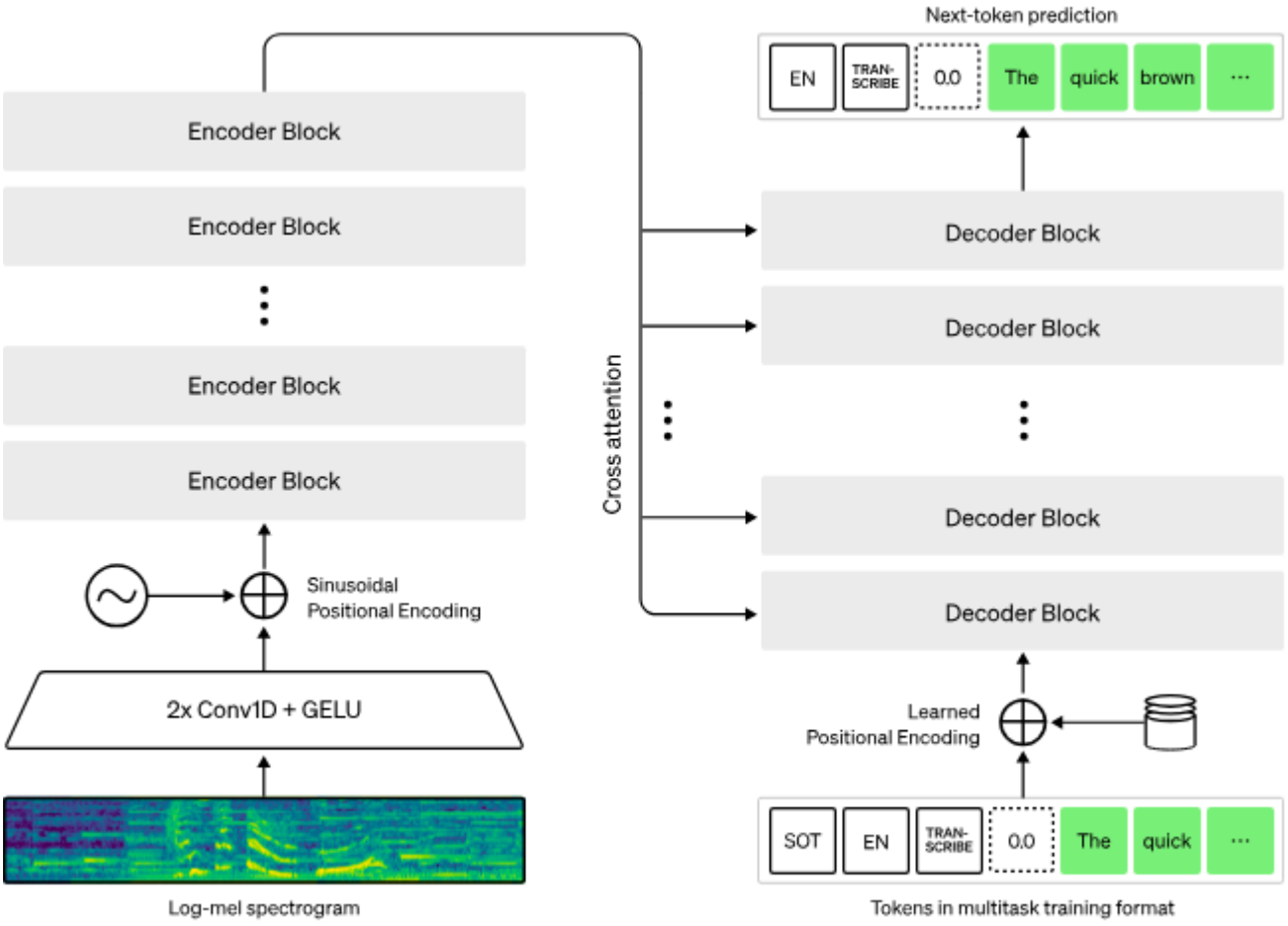

image-168.png

架构详解

- **处理架构**：
  1. **Encoder**：接收 log-mel spectrogram，将其编码为连续的 hidden-states 序列，负责提取输入语音的深层声学特征。

```
- [!] seq2seq 的 input 通常为 spectrogram 但是直接传入 audio waveforms 也是可以的
```

  2. **Decoder**：接收 Encoder 的输出特征，通过自回归 (Autoregressive) 方式，逐个预测并生成目标文本 Token。
- **Decoder 的核心机制**：
  - **Cross-attention (交叉注意力)**：这是 Decoder 连接 Encoder 的桥梁。它允许 Decoder 在生成文本时，动态关注并提取 Encoder 产生的音频特征。
  - **Causal attention (因果注意力)**：在自回归生成过程中，严格掩码 (Mask) 未来的信息，确保 Decoder 只能基于已生成的历史 Token 预测下一个 Token。
- **输出特征**：
  - **Token 粒度**：Seq2Seq 模型通常复用成熟 NLP 模型的 tokenizer（例如 GPT-2 tokenizer，词表超 50k）。因此，其输出序列长度大幅短于 CTC 输出
- **训练与评估**：
  - **损失函数与指标**：最终层输出概率分布，通常使用 Cross-entropy loss 进行端到端训练。在推理阶段常结合 **Beam search** 优化生成质量。其核心评估指标为 **WER (Word Error Rate, 词错误率)**。

  ℹ **Beam search**

Text-to-speech (TTS)

- **处理架构**：本质上是 ASR 的逆向映射过程。
  1. **Encoder**：接收文本 Token 序列，提取文本的隐藏状态特征。
  2. **Decoder**：应用交叉注意力机制结合文本特征，预测并生成 spectrogram 序列。
- **生成机制与终止条件**：
  - Decoder 的初始输入是一个**全零的、长度为 1** 的 spectrogram（start token）。
  - **终止预测机制 (Stop Prediction)**：由于输出长度未知，模型（如 SpeechT5）会强制 Decoder 输出一个平行的概率序列，用于预测“当前时间步是否为最后一帧”。当此概率超过设定阈值（如 0.5）时，终止自回归循环。
- **Post-processing**：
  - **Post-net**：由多个卷积层组成，用于对 Decoder 生成的初始 spectrogram 进行细化和降噪。
  - **Vocoder**：如同上文所述，还原 waveform

- **评估**：
  - **MOS (Mean Opinion Score, 平均主观意见分)** 进行评估

Audio classification architectures

- **Audio classification** 的核心目标：为输入的音频预测一个或多个类型标签
- **两种不同的分类粒度**：
  - **Sequence-level classification（序列级）**：输入整段音频，输出一个**全局**的单一类别标签。
  - **Frame-level classification（帧级）**：输入整段音频，为序列中的**每一个音频帧**（通常是每 20 毫秒的切片）预测一个标签。最终输出的是一个类别概率的序列。

Classification using spectrograms

- 类似于 Vit 的多模态思想，不多赘述。

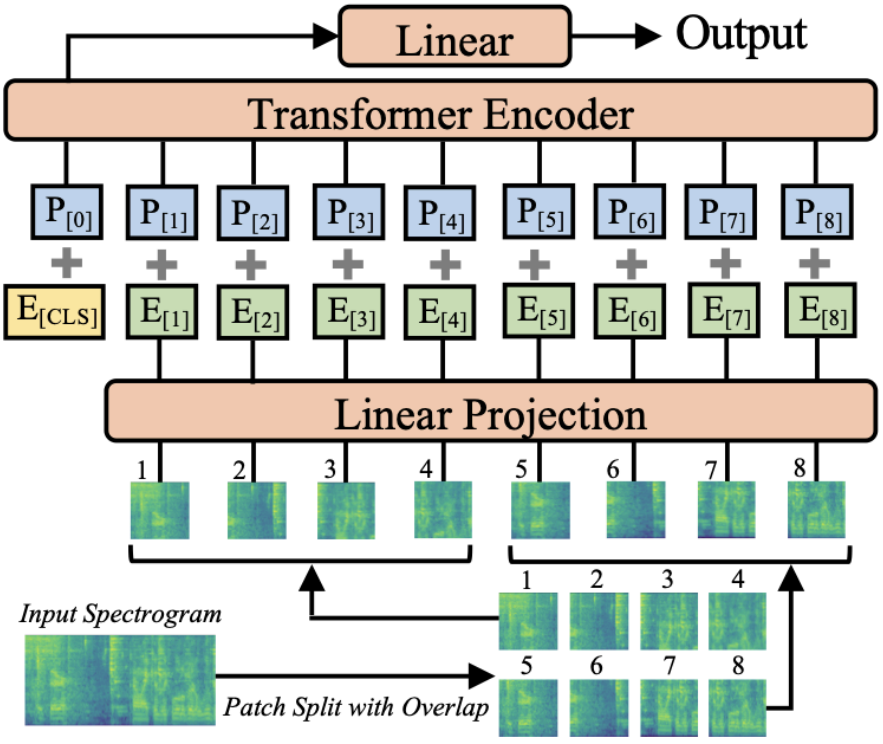

image-169.png

- **本质差异**：虽然我们将频谱图当作图片处理，但它们本质不同。
  - 普通图像具有**平移不变性**：图像内容上下移动，表达的含义不变。
  - 频谱图**不具备**平移不变性：纵轴代表频率，将特征在纵轴上平移，意味着彻底改变了声音的音高和本质。

Any transformer can be a classifier

- 还是通过 encoder 来提取特征，只是将原本用于特定任务的输出层替换为一个新的分类层

- **全局序列分类**：模型会对输出的所有 hidden-states **求平均值 (Mean)**，然后将这个融合了全局信息的单一向量送入分类层，输出一个最终的概率分布。